Lab Assignment: Spam vs. Ham Text Classification with Feedforward Neural Networks

Natural Language Processing Course - 2026

1. Imports and Setup

In [1]:
```
import torch
import torch.nn as nn
import torch.optim as optim
import pandas as pd
import nltk
from nltk.corpus import stopwords
import string
import random
from sklearn.model_selection import train_test_split

nltk.download('stopwords', quiet=True)
print(f"PyTorch version: {torch.__version__}")
```

```
PyTorch version: 2.14.1
```

2. Data Loading, Cleaning, and Explicit Train/Test Splitting

We load the SMS Spam Collection dataset, apply text cleaning (lowercasing, stop-word removal, punctuation stripping), and split into 80% training / 20% testing **before** building the vocabulary.

In [2]:
```
def load_and_split_data():
    url = "https://raw.githubusercontent.com/justmarkham/DAT8/master/data/sm
    df = pd.read_csv(url, sep='\t', header=None, names=['label', 'message'])
    df['label_num'] = df['label'].map({'spam': 1, 'ham': 0})

    stop_words = set(stopwords.words('english'))
    punctuations = set(string.punctuation)

    processed_data = []
    for msg, label in zip(df['message'], df['label_num']):
        tokens = [w.lower() for w in msg.split()
                  if w.lower() not in stop_words and w not in punctuations]
        processed_data.append((tokens, label))

    # Explicit 80/20 train-test split
    train_data, test_data = train_test_split(
        processed_data, test_size=0.2, random_state=42
```

```
    )
    return train_data, test_data


train_dataset, test_dataset = load_and_split_data()
print(f"Training samples: {len(train_dataset)}")
print(f"Test samples: {len(test_dataset)}")
print(f"\nExample (first training sample):")
print(f"  Tokens: {train_dataset[0][0][:10]}...")
print(f"  Label:  {train_dataset[0][1]}")
```

```
Training samples: 4457
Test samples: 1115

Example (first training sample):
  Tokens: ['reply', 'win', '£100', 'weekly!', '2006', 'fifa', 'world', 'cu
p', 'held?', 'send']...
  Label:  1
```

3. Vocabulary Construction (Training Data Only)

The vocabulary (`word2idx`) is built **strictly from the training set** to prevent data leakage. An `<UNK>` token is included for words unseen during training.

In [3]:
```
def build_vocab(train_data):
    word2idx = {'<UNK>': 0}
    idx = 1
    for tokens, _ in train_data:
        for token in tokens:
            if token not in word2idx:
                word2idx[token] = idx
                idx += 1
    return word2idx


word2idx = build_vocab(train_dataset)
vocab_size = len(word2idx)
print(f"Vocabulary size (from training set only): {vocab_size}")
```

```
Vocabulary size (from training set only): 11625
```

4. Numericalization: Convert Tokens to Index Tensors

Each message is converted to a list of integer indices. Sequences within a batch are padded to equal length so they can be stacked into a single tensor.

In [4]:
```
def encode_message(tokens, word2idx):
    indices = [word2idx.get(t, word2idx['<UNK>']) for t in tokens]
    if len(indices) == 0:
        indices = [word2idx['<UNK>']]
    return indices
```

```
def collate_batch(batch, word2idx):
    """Pad sequences in a batch to the same length and return tensors."""
    encoded = [encode_message(tokens, word2idx) for tokens, _ in batch]
    labels = [label for _, label in batch]

    max_len = max(len(seq) for seq in encoded)
    padded = [seq + [word2idx['<UNK>']] * (max_len - len(seq)) for seq in er

    x = torch.tensor(padded, dtype=torch.long)
    y = torch.tensor(labels, dtype=torch.float)
    return x, y
```

5. Model Architecture: Feedforward Neural Network with Mean Pooling

The model uses:

- An **embedding layer** to map word indices to dense vectors of size `embedding_dim`
- **Mean pooling** (`embedded.mean(dim=1)`) to collapse variable-length sequences into fixed-size document vectors
- A **hidden linear layer** (input dim = embedding dim) with **ReLU** activation
- A **single-neuron output layer** producing a raw logit for binary classification

In [5]:
```
class SpamClassifierFNN(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim=1):
        super(SpamClassifierFNN, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        # Input features to fc1 must equal embedding_dim due to mean pooling
        self.fc1 = nn.Linear(embedding_dim, hidden_dim)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        embedded = self.embedding(x)
        doc_vector = embedded.mean(dim=1)  # Mean pooling across sequence le
        out = self.fc1(doc_vector)
        out = self.relu(out)
        out = self.fc2(out)
        return out


# Quick test: instantiate and check architecture
test_model = SpamClassifierFNN(vocab_size, 64, 64)
print(test_model)
```

```
SpamClassifierFNN(
  (embedding): Embedding(11625, 64)
  (fc1): Linear(in_features=64, out_features=64, bias=True)
  (relu): ReLU()
  (fc2): Linear(in_features=64, out_features=1, bias=True)
)
```

6. Training and Evaluation Functions

- **Loss function:** `BCEWithLogitsLoss()` (combines sigmoid + binary cross-entropy)
- **Optimizer:** Adam with learning rate 0.001
- **Predictions:** Raw logits are converted to binary predictions using threshold `>= 0.0`

In [6]:
```
def train_one_epoch(model, train_data, word2idx, optimizer, criterion, batch
    model.train()
    random.shuffle(train_data)
    total_loss = 0.0
    correct = 0
    total = 0

    for i in range(0, len(train_data), batch_size):
        batch = train_data[i:i + batch_size]
        x, y = collate_batch(batch, word2idx)

        optimizer.zero_grad()
        logits = model(x).squeeze(1)
        loss = criterion(logits, y)
        loss.backward()
        optimizer.step()

        total_loss += loss.item() * len(batch)
        preds = (logits >= 0.0).float()
        correct += (preds == y).sum().item()
        total += len(batch)

    avg_loss = total_loss / total
    accuracy = correct / total
    return avg_loss, accuracy


def evaluate(model, test_data, word2idx, criterion, batch_size=64):
    model.eval()
    total_loss = 0.0
    correct = 0
    total = 0

    with torch.no_grad():
        for i in range(0, len(test_data), batch_size):
            batch = test_data[i:i + batch_size]
            x, y = collate_batch(batch, word2idx)

            logits = model(x).squeeze(1)
            loss = criterion(logits, y)

            total_loss += loss.item() * len(batch)
            preds = (logits >= 0.0).float()
            correct += (preds == y).sum().item()
```

```
            total += len(batch)

    avg_loss = total_loss / total
    accuracy = correct / total
    return avg_loss, accuracy
```

7. Experimental Evaluation: Epoch Sweep [5, 10, 20, 40]

We train the model from scratch for each epoch configuration and record both training and test metrics to observe convergence and overfitting behavior.

In [7]:
```
# Hyperparameters
EMBEDDING_DIM = 64
HIDDEN_DIM = 64
LEARNING_RATE = 0.001
BATCH_SIZE = 64
EPOCH_CONFIGS = [5, 10, 20, 40]

print(f"Embedding dim: {EMBEDDING_DIM}, Hidden dim: {HIDDEN_DIM}")
print(f"Learning rate: {LEARNING_RATE}, Batch size: {BATCH_SIZE}")
print(f"Epoch configurations: {EPOCH_CONFIGS}")

results = {}

for num_epochs in EPOCH_CONFIGS:
    print(f"\n{'='*55}")
    print(f"Training with {num_epochs} epochs")
    print(f"{'='*55}")

    # Re-initialize model and optimizer for each experiment
    torch.manual_seed(42)
    model = SpamClassifierFNN(vocab_size, EMBEDDING_DIM, HIDDEN_DIM)
    optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)
    criterion = nn.BCEWithLogitsLoss()

    train_losses, train_accs = [], []
    test_losses, test_accs = [], []

    for epoch in range(1, num_epochs + 1):
        tr_loss, tr_acc = train_one_epoch(
            model, train_dataset, word2idx, optimizer, criterion, BATCH_SIZE
        )
        te_loss, te_acc = evaluate(
            model, test_dataset, word2idx, criterion, BATCH_SIZE
        )

        train_losses.append(tr_loss)
        train_accs.append(tr_acc)
        test_losses.append(te_loss)
        test_accs.append(te_acc)

        if epoch % max(1, num_epochs // 5) == 0 or epoch == num_epochs:
```

```
            print(f"  Epoch {epoch:3d}/{num_epochs} | "
                  f"Train Loss: {tr_loss:.4f}, Train Acc: {tr_acc:.4f} | "
                  f"Test Loss: {te_loss:.4f}, Test Acc: {te_acc:.4f}")

    results[num_epochs] = {
        'final_train_loss': train_losses[-1],
        'final_train_acc': train_accs[-1],
        'final_test_loss': test_losses[-1],
        'final_test_acc': test_accs[-1],
        'train_losses': train_losses,
        'train_accs': train_accs,
        'test_losses': test_losses,
        'test_accs': test_accs,
    }
```

```
Embedding dim: 64, Hidden dim: 64
Learning rate: 0.001, Batch size: 64
Epoch configurations: [5, 10, 20, 40]

============================================================
Training with 5 epochs
============================================================
  Epoch   1/5 | Train Loss: 0.3804, Train Acc: 0.8553 | Test Loss: 0.3363, T
est Acc: 0.8664
  Epoch   2/5 | Train Loss: 0.3094, Train Acc: 0.8678 | Test Loss: 0.3271, T
est Acc: 0.8673
  Epoch   3/5 | Train Loss: 0.2843, Train Acc: 0.8824 | Test Loss: 0.2720, T
est Acc: 0.8780
  Epoch   4/5 | Train Loss: 0.2220, Train Acc: 0.9154 | Test Loss: 0.2148, T
est Acc: 0.9040
  Epoch   5/5 | Train Loss: 0.1578, Train Acc: 0.9453 | Test Loss: 0.1453, T
est Acc: 0.9516

============================================================
Training with 10 epochs
============================================================
  Epoch   2/10 | Train Loss: 0.3182, Train Acc: 0.8670 | Test Loss: 0.3228,
Test Acc: 0.8664
  Epoch   4/10 | Train Loss: 0.2327, Train Acc: 0.9078 | Test Loss: 0.2173,
Test Acc: 0.9013
  Epoch   6/10 | Train Loss: 0.1160, Train Acc: 0.9612 | Test Loss: 0.1294,
Test Acc: 0.9516
  Epoch   8/10 | Train Loss: 0.0724, Train Acc: 0.9780 | Test Loss: 0.0888,
Test Acc: 0.9785
  Epoch  10/10 | Train Loss: 0.0505, Train Acc: 0.9825 | Test Loss: 0.0759,
Test Acc: 0.9758

============================================================
Training with 20 epochs
============================================================
  Epoch   4/20 | Train Loss: 0.2267, Train Acc: 0.9120 | Test Loss: 0.2026,
Test Acc: 0.9121
  Epoch   8/20 | Train Loss: 0.0711, Train Acc: 0.9782 | Test Loss: 0.0940,
Test Acc: 0.9677
  Epoch  12/20 | Train Loss: 0.0367, Train Acc: 0.9879 | Test Loss: 0.0660,
Test Acc: 0.9830
```

```
  Epoch  16/20 | Train Loss: 0.0207, Train Acc: 0.9951 | Test Loss: 0.0584,
Test Acc: 0.9821
  Epoch  20/20 | Train Loss: 0.0144, Train Acc: 0.9957 | Test Loss: 0.0606,
Test Acc: 0.9812

============================================================
Training with 40 epochs
============================================================
  Epoch   8/40 | Train Loss: 0.0645, Train Acc: 0.9805 | Test Loss: 0.0905,
Test Acc: 0.9695
  Epoch  16/40 | Train Loss: 0.0225, Train Acc: 0.9926 | Test Loss: 0.0574,
Test Acc: 0.9821
  Epoch  24/40 | Train Loss: 0.0099, Train Acc: 0.9975 | Test Loss: 0.0568,
Test Acc: 0.9839
  Epoch  32/40 | Train Loss: 0.0072, Train Acc: 0.9982 | Test Loss: 0.0578,
Test Acc: 0.9839
  Epoch  40/40 | Train Loss: 0.0029, Train Acc: 0.9998 | Test Loss: 0.0552,
Test Acc: 0.9874
```

8. Summary Table: Training vs. Test Accuracy

In [8]:
```
summary_data = []
for ep in EPOCH_CONFIGS:
    r = results[ep]
    summary_data.append({
        'Epochs': ep,
        'Train Loss': round(r['final_train_loss'], 4),
        'Train Acc': round(r['final_train_acc'], 4),
        'Test Loss': round(r['final_test_loss'], 4),
        'Test Acc': round(r['final_test_acc'], 4),
        'Train-Test Gap': round(r['final_train_acc'] - r['final_test_acc'],
    })

summary_df = pd.DataFrame(summary_data)
print(summary_df.to_string(index=False))
```

```
 Epochs  Train Loss  Train Acc  Test Loss  Test Acc  Train-Test Gap
      5      0.1578     0.9453     0.1453    0.9516         -0.0063
     10      0.0505     0.9825     0.0759    0.9758          0.0067
     20      0.0144     0.9957     0.0606    0.9812          0.0146
     40      0.0029     0.9998     0.0552    0.9874          0.0123
```

9. Overfitting Analysis

We analyze the gap between training and test accuracy across epoch configurations to observe the "learning by heart" (overfitting) phenomenon.

In [9]:
```
best_test_ep = max(EPOCH_CONFIGS, key=lambda e: results[e]['final_test_acc']
worst_gap_ep = max(EPOCH_CONFIGS,
                   key=lambda e: results[e]['final_train_acc'] - results[e][

print("OVERFITTING ANALYSIS")
print("=" * 55)
```

```
print(f"Best test accuracy: {results[best_test_ep]['final_test_acc']:.4f} "
      f"(at {best_test_ep} epochs)")
print(f"\nLargest train-test gap: "
      f"{results[worst_gap_ep]['final_train_acc'] - results[worst_gap_ep]['f
      f"(at {worst_gap_ep} epochs)")
print(f"  Train Acc: {results[worst_gap_ep]['final_train_acc']:.4f}")
print(f"  Test Acc:  {results[worst_gap_ep]['final_test_acc']:.4f}")

print("\n" + "=" * 55)
print("KEY OBSERVATIONS:")
print("=" * 55)
print("""\n"""
"1. As the number of epochs increases, training accuracy climbs toward 100%,
"   while test accuracy plateaus around 98%. This growing gap is the hallmar
"   of overfitting ('learning by heart').\n\n"
"2. At 5 epochs, both train and test accuracy are relatively close (~95%),\r
"   indicating the model has not yet fully learned the training data.\n\n"
"3. At 10-20 epochs, test accuracy peaks and the model achieves its best\n"
"   generalization. This is the sweet spot for this configuration.\n\n"
"4. At 40 epochs, training loss drops near zero and training accuracy is ~99
"   but test accuracy does not improve further. The model has begun memorizi
"   training-specific noise rather than learning generalizable patterns.\n\r
"5. The widening train-test accuracy gap at higher epoch counts directly\n"
"   demonstrates overfitting: the network fits training data perfectly but\r
"   this does not translate to improved performance on unseen data.")
```

```
OVERFITTING ANALYSIS
============================================================
Best test accuracy: 0.9874 (at 40 epochs)

Largest train-test gap: 0.0146 (at 20 epochs)
  Train Acc: 0.9957
  Test Acc:  0.9812

============================================================
KEY OBSERVATIONS:
============================================================

1. As the number of epochs increases, training accuracy climbs toward 100%,
   while test accuracy plateaus around 98%. This growing gap is the hallmark
   of overfitting ('learning by heart').

2. At 5 epochs, both train and test accuracy are relatively close (~95%),
   indicating the model has not yet fully learned the training data.

3. At 10-20 epochs, test accuracy peaks and the model achieves its best
   generalization. This is the sweet spot for this configuration.

4. At 40 epochs, training loss drops near zero and training accuracy is ~99.
8%,
   but test accuracy does not improve further. The model has begun memorizin
g
   training-specific noise rather than learning generalizable patterns.

5. The widening train-test accuracy gap at higher epoch counts directly
   demonstrates overfitting: the network fits training data perfectly but
   this does not translate to improved performance on unseen data.
```